

Popular Performance Monitoring Tools for Container Technology

Containers as microservice-based technology have a significant role in virtualising cloud applications. This article takes a quick look at the most popular performance monitoring tools for container technology that can ease the task of IT admins.

Containerisation is a technology to virtualise applications in a lightweight way that consumes less resources and time. This has led to the development of various container technologies like LXC, Docker and RKT. All these technologies work on the same principle — that the applications can share their host OS kernel and contains only appropriate binaries and libraries, making it smaller in size compared to a virtual machine.

Why monitoring of containers is important

The monitoring of containers plays a significant role for developers because it gives a quick overview of the running applications and ensures that containers are meeting their expected goals. This helps to catch problems early and resolve issues quickly.

Popular tools for performance monitoring of containers

As containers run in their own namespaces, traditional Linux monitoring tools such as `top`, `ps`, `tcpdump` and `lsof`, from the host system, do not help to monitor the activity happening within a container. A proper understanding of the following tools will help researchers, practitioners and developers efficiently monitor the applications deployed on the

container virtualisation platform. So let's look at these popular container monitoring tools and their installation steps.

Docker Stats

To monitor resource usage of Docker containers, the simple solution is the `Docker stats` command. It is an open source default API available in the Docker daemon, and provides resource usage statistics of a running Docker container in terms of CPU, RAM, network, and block I/O usage. The following command can be used to get performance metrics of a running Docker container, as depicted in Figure 1:

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O
DS6d6747a01757	c1	0.00%	804KiB / 3.729GiB	0.02%	33.3kB / 0B

Figure 1: Container monitoring with the `Docker stats` command

```
root@server3:~# sudo docker run \
> --volume=/:/rootfs:ro \
> --volume=/var/run:/var/run:ro \
> --volume=/sys:/sys:ro \
> --volume=/var/lib/docker:/var/lib/docker:ro \
> --volume=/dev/disk:/dev/disk:ro \
> --publish=8080:8080 \
> --detach=true \
> --name=cadvisor \
> gcr.io/google-containers/cadvisor:latest
Unable to find image 'gcr.io/google-containers/cadvisor:latest' locally
latest: Pulling from google-containers/cadvisor
9d48c3bd43c5: Pull complete
f7d6cbe0ad90: Pull complete
15f5311b080f: Pull complete
Digest: sha256:46d4d730ef886aaece9e0a65a912564cab0303cf88718d82b3df84d3add6885c
Status: Downloaded newer image for gcr.io/google-containers/cadvisor:latest
6cd8c05e179ddb530e35f8683a8f88f9d5892f324d0954ecd655ecfc9b8793b3
root@server3:~#
```

Figure 2: Installing the cAdvisor container monitoring tool

`$ docker stats container name`

cAdvisor

cAdvisor stands for container advisor and was created by Google in 2014. It is also an open source tool to monitor Docker containers. However, compared to Docker Stats, which is based on the command line interface (CLI), cAdvisor also provides a GUI for viewing API information. In cAdvisor, the isolation of the shared resources used by multiple container applications is based on `lmctfy`'s API. To set up and run cAdvisor, execute the following commands, as shown in Figure 2: